Which Model Family Should Run on the Sensor?
Leakage-Aware, End-to-End Accuracy–Energy–Memory Analysis of TinyML Bearing-Fault Diagnosis on a Cortex-M4

Agnibha Sarkar, Abhinav Yadaram and Hridayesh Kundu
Department of Electrical and Electronics Engineering
Birla Institute of Technology and Science, Pilani, K. K. Birla Goa Campus
Goa, India

***Abstract*—Microcontroller studies of bearing-fault diagnosis usually report one model, scored on a split that reuses the same physical bearing for training and testing, with an energy figure that leaves out feature extraction. We compare four model families (decision trees, random forests, MLPs and tiny 1-D CNNs in float32 and int8) as 88 deployable configurations, evaluated on the CWRU dataset under three split protocols: leaky, recording-wise and bearing-disjoint. Every configuration is trained over 8 folds and 5 seeds (3,520 host runs). Before the measurement hardware arrived, we built a virtual Cortex-M4F. It runs the exact deployable code (the C front end on CMSIS-DSP, emlearn trees, and TensorFlow Lite Micro with CMSIS-NN or reference kernels) inside QEMU and counts instructions exactly, using a cycle model taken from the Cortex-M4 technical reference manual. On it, the C front end matches the Python reference to 9.6×10^{-5} and every network reproduces the host predictions on the 100 fixed test vectors. Leakage changes the answer. The best model scores 1.000 macro-F1 under the leaky split but 0.742 on unseen bearings, and the family ranking changes between protocols (Kendall τ = 0.67 between the P1 and P3 hypervolume rankings). The front end dominates the cost of feature-based models: 124.0 k cycles for R2 features against 1,589 cycles for the largest tree, 95.7 % or more of their end-to-end work. Three deployment pitfalls surfaced and were fixed before any board measurement: FFT bands below float32 precision, soft- versus hard-voting forests, and desktop int8 kernels that round differently from the microcontroller's. Because no family exceeded 0.623 on unseen bearings, we added a physics-informed envelope-spectrum front end (R4), which raises the best bearing-disjoint macro-F1 to 0.742. A simulated measurement campaign that runs synthetic INA219 samples through the logger's own integration code predicts a session-to-session CV of 0.46 % and about 16 hours of board time. Energies here are chip-level estimates; INA219 measurements replace them in the final version.**

***Index Terms*—TinyML, bearing fault diagnosis, CWRU, data leakage, Pareto frontier, microcontroller, energy, TensorFlow Lite Micro**

I. Introduction

Bearings are the most common single cause of failure in large electric motors, and unplanned downtime is expensive. A vibration sensor can detect bearing damage early, but raw vibration sampled at 12 kHz and 16 bit (192 kbit/s) is about 35 times the fastest LoRaWAN data rate. The classifier therefore has to run on the sensor itself: a microcontroller with a few hundred kilobytes of RAM and a battery expected to last years.

Several model families fit such a device. Decision trees and random forests need only a few hundred bytes and run in microseconds, but they require a feature front end (statistics, an FFT). Small MLPs take the same features. Tiny 1-D CNNs read the raw window, avoid the front end, and do the most arithmetic. Int8 quantization makes CNNs about four times smaller and, with CMSIS-NN kernels, much faster. An engineer who has to pick one family for a new sensor finds little evidence in the literature, for three reasons:

1) *One model, one number.* Microcontroller bearing papers report a single model's accuracy and latency; families are not compared on one board, and energy is rarely measured [1], [2].
2) *Only inference is counted.* The feature front end is left out, although on this very board it was estimated at 52–71× a raw-input model's energy [3].
3) *Evaluation leaks.* CWRU has one physical bearing per fault class. Random or load-wise splits test on a bearing the model has already seen, and independent-bearing splits cut CNN accuracy roughly in half [4], [5].

This paper reports the part of the study that can be done before the energy-measurement rig is built. Its contributions are:

- a complete, leakage-checked host evaluation of 88 configurations over three protocols, eight folds and four noise levels;
- a *virtual Cortex-M4* harness that executes the deployable code and gives exact instruction counts, a cycle estimate, flash, arena and stack use for every model, and passes the feature-parity and prediction-agreement gates in emulation;
- preliminary accuracy–energy–memory frontiers with hypervolume, bootstrap membership and rank-reversal statistics, a battery-life model and a first selection rubric;
- three deployment pitfalls, each found by the pipeline's own gates and fixed;
- a physics-informed envelope front end (R4) for unseen bearings, verified in C against the Python reference;
- a virtual measurement campaign, run through the logger's own C code, that sizes the rig's error budget and the campaign duration before the hardware arrives, and an interactive model selector built from the results.

The code, data manifests, trained models and all results are available at https://github.com/agnibha11/TinyML-Pareto.

II. Background and Related Work

Leakage on CWRU. Hendriks *et al.* showed that the usual load-wise split keeps the same bearing in training and test, and that grouping by fault size instead drops the best model from 95.0 % to 53.1 % accuracy [4]. Vieira *et al.* used bearing-wise splits across four datasets; on CWRU, a random forest on handcrafted features (85.1 % macro AUROC) beat a CNN (74.5 %) [5]. Neither study deploys a model.

Measured TinyML cost. MLPerf Tiny standardised energy measurement with GPIO-gated windows of at least 10 s, but excludes pre-processing [6]. Jain *et al.* measured energy–accuracy fronts of neural networks on Cortex-M0+/M4/M7 [7]. Bartoli *et al.* measured end-to-end keyword spotting and found pre-inference to be 59–70 % of the energy [8], [9]. Zhang *et al.* characterised power and performance of TinyML systems on four MCUs [10].

Bearing diagnosis on MCUs. Deployed CWRU classifiers report 98.6–99.9 % accuracy with a single CNN, with splits that reuse bearings or with inference-only energy [1], [2]. A concurrent dataset release measures 144 int8 networks for CWRU on an ESP32-S3 with a calibrated INA219 chain [11]; it covers neural networks only (1-D CNNs, depthwise-separable CNNs and MLPs). To our knowledge, no study compares trees, MLPs and float/int8 CNNs on one Cortex-M4 with front-end and inference cost separated, or asks whether the frontier changes once evaluation leakage is removed.

III. Method

A. Data

We use the drive-end accelerometer of the CWRU bearing data set [12], [13]: normal and inner-race (IR), ball (B) and outer-race (OR, 6 o'clock) faults of 0.007, 0.014 and 0.021 in, at loads 0–3 HP, i.e. 40 recordings and 10 classes. Normal recordings (48 kHz) are decimated to 12 kHz. Every recording then passes the same zero-phase 255-tap FIR low-pass at 5 kHz, because the recorder's anti-alias roll-off otherwise exists only in the fault files and a single band feature separates normal from all faults. Recordings are cut into 1024-sample windows (85 ms) at stride 512: 9,346 windows, 4,683 of them non-overlapping (401 s of signal).

B. Split protocols

All splits are generated from the window manifest and checked by assertions (Table I). **P1 (leaky)** draws a stratified random 70/15/15 split of all overlapping windows, as most papers do; 98.1 % of its test windows share samples with a training or validation window. **P2 (recording-wise)** holds out one load (4 folds, 10 classes): no window leaks, but every test bearing was seen at other loads. **P3 (bearing-disjoint)** holds out one fault size (3 folds) and predicts the fault type (4 classes), so every fault test bearing is unseen; the single normal bearing is split by load (train 0–2 HP, test 3 HP). P2 and P3 test windows are non-overlapping. Validation sets (15 % of training windows) are used only for early stopping.

TABLE I
Split protocols (window counts, fold 0).

Protocol	Classes	Folds	Train / val	Test
P1 leaky	10	1	6,542 / 1,402	1,402
P2 recording-wise	10	4	6,033 / 1,065	1,126
P3 bearing-disjoint	4	3	5,326 / 941	1,542

C. Input representations

R0 is the raw window, normalised by two global training-set scalars (amplitude carries fault-size information, so windows are not normalised individually). **R1** has 10 time-domain statistics: mean, RMS, standard deviation, peak, peak-to-peak, crest, kurtosis, skewness, shape and impulse factor. **R2** is R1 plus the log power of 14 bands of 375 Hz from a Hann-windowed 1024-point FFT, covering 0–5.25 kHz. **R3** is the log magnitude of FFT bins 1–256. **R4**, added after the first sweep (Section IV-C), is the classical envelope analysis of bearing diagnostics [13], [14] reduced to a fixed vector: the 2–5 kHz band of the un-windowed 1024-point FFT is shifted to baseband, a 256-point inverse FFT gives the envelope (at 3 kHz), and the log magnitude of its Hann-windowed spectrum at 11.7–375 Hz (32 bins, divided by their mean) captures the impact rates of outer-race, inner-race and ball faults (about 3.6, 5.4 and 4.7 times the shaft speed, 70–165 Hz here). These rates depend on where the fault is, not on its size, and no shaft-speed input is needed. R4 also carries R1's statistics except the window mean (Section IV-C): 41 features. Features are z-scored with training statistics of each fold; the deployed models use P2 fold 0.

D. Model families and sweep

Table II lists the 88 configurations. CNN blocks are Conv (kernel 9 and stride 2 in the first block, kernel 3 afterwards), BatchNorm, ReLU and MaxPool(2), with $16w$ channels doubling per block and capped at $64w$, followed by global average pooling and a dense softmax layer. They are written as Conv2D ($k \times 1$) because the quantization-aware training toolkit rejects Conv1D. NNs use Adam (lr 10^{-3}, batch 64), at most 100 epochs and early stopping on validation loss (patience 10). Int8 models use full-integer post-training quantization (PTQ) with 500 fixed training windows; QAT fine-tunes the float model for 20 epochs at lr 10^{-4}. NN scores come from the exported `.tflite` file run in the TFLite interpreter, never from Keras. Each configuration is trained on every fold of every protocol with 5 seeds; scores are macro-F1, reported as mean over folds and seeds. The deployed instance is P2, fold 0, seed 0.

E. Virtual Cortex-M4 harness

The Arduino Nano 33 BLE (nRF52840, Cortex-M4F, 64 MHz) and the INA219 rig were not yet available, so every deployable artefact was compiled for Cortex-M4F (`arm-none-eabi-gcc` 13.2, FPv4-SP with the softfp ABI, `-Os`, as in the Arduino core) and run on QEMU's `mps2-an386` machine [15]. Three images hold, respectively,

TABLE II
DESIGN MATRIX (88 MODEL IDS; R4 ROWS ADDED AFTER THE FIRST SWEEP).

Family	Input	Sizes	Precision	IDs
Decision tree	R1, R2, R4	depth 4, 6, 8, 12	fp32	12
Random forest	R1, R2, R4	5, 10, 25 trees; depth 6, 10	fp32	18
MLP	R2, R4	16, 32, 64, 32-32, 64-64	fp32, int8	20
MLP	R3	32, 64, 128	fp32, int8	6
1-D CNN	R0	width 0.25–2; 2 or 4 blocks	fp32, int8, QAT	24
CNN ablation	R0	the same 8	int8, ref. kernels	8

all emlearn trees [16]; all networks on TensorFlow Lite Micro [17] with CMSIS-NN kernels [18]; and the int8 CNNs on reference kernels (the kernel ablation). The images contain the same C front end (CMSIS-DSP `arm_rfft_fast_f32`) that the board firmware will use, the explicit five-operator resolver of the deployment, and the 5 stored windows and 100 test vectors of the handoff package.

A TCG plugin counts every executed instruction between two marker functions that bracket the measured phases. Phases follow the board contract: *front end* (features and z-score, or R0 normalisation) and *inference* (input quantisation, `Invoke()` or `predict()`, argmax). Each instruction of the ELF is also assigned a cost from the Cortex-M4 TRM timing tables [19] (ALU and MAC 1 cycle, single load/store 2, LDM/STM $1 + N$, branches 2–3, VDIV and VSQRT 14, VMLA 3), so the plugin reports an estimated cycle count. Flash wait states and bus contention are not modelled, so these are *estimates to be calibrated* against the board's DWT cycle counter. The harness also reports flash (symbol sizes), the TFLM arena and the peak stack (painted-stack method). It replays the VERIFY command on the 100 test vectors and dumps the front-end outputs for the feature-parity gate.

F. Energy estimate and analysis

Until the INA219 data exist, energy is estimated as $E = (\text{cycles}/f)\, P_{\text{act}}$ with $f = 64\,\text{MHz}$ and $P_{\text{act}} = 9.98\,\text{mW}$. This corresponds to $52\,\mu\text{A}/\text{MHz}$ at 3 V, the nRF52840's CoreMark-from-flash figure with DC/DC [20]. It is a chip-level, cycle-proportional estimate. The Nano's Mbed configuration keeps the LDO regulator, which draws roughly twice the DC/DC current at the chip. That factor is the same for every model, so it scales the energy axis without changing any ranking. The board-level measurements will add the regulator, USB PHY and idle overhead that this estimate omits. Peak RAM is the sum of arena, feature buffers and peak stack. Objectives, all minimised, are error ($1-$macro-F1), $\log_{10}$ energy and $\log_2$ peak RAM. For each protocol and for inference-only and end-to-end energy we compute the non-dominated set, the hypervolume of each family after min–max normalisation (reference point 1.1; pymoo [21]), each family's exclusive contribution, and the probability that a model is non-dominated over 1000 paired bootstrap resamples of the test windows. Family rankings are compared with Kendall's τ. Battery life uses $P_{\text{avg}} = r\,E + P_{\text{sleep}}$ and a CR2032 cell (225 mAh, 3 V); the crossover rate is $r^* = P_{\text{sleep}}/E$.

G. Virtual measurement campaign

To plan the hardware phase, the campaign of the board contract is replayed on a simulated rig. For every model, the window schedule (IDLE, then RUN, FRONT and E2E with N chosen so that each window lasts at least 10 s) follows from its latency on the virtual Cortex-M4. The board current in each window is synthesised from a stated power model: 6 mA at 5 V in the idle loop, 4.9 mA more while the CPU runs (the nRF52840's active current through the board's buck converter), 0.3 mA of per-sample noise, and baseline drift within and between windows and sessions. A small C program quantises the current exactly as the INA219 registers do (10 μV shunt and 4 mV bus LSBs, $\pm$40 mV range, 532 µs conversions, offset and 0.5 % gain error) and integrates it with the logger's own C core, the code that will run on the ESP32; the measurement script's parser reads the resulting lines. Energy per inference is then recovered as the analysis will do it, $\Delta E = (\bar{P}_{\text{RUN}} - \bar{P}_{\text{IDLE}})\, T_{\text{RUN}}/N$, over three sessions and 30 Monte-Carlo replications, and compared with the injected truth. The power-model values are assumptions; the first rig sessions (gate M3) replace them. What the simulation establishes is which effects dominate the error, and how long the campaign will take.

IV. PRELIMINARY RESULTS

A. Deployment correctness before hardware

Feature parity (gate M2). On the virtual Cortex-M4, the C front end reproduces `ml/features.py` on the 100 fixed test vectors to a maximum normalised error of 1.4×10^{-5} (R1), 1.4×10^{-5} (R2) and 9.6×10^{-5} (R3), all well inside the 10^{-3} gate. **Prediction agreement.** Replaying the board's VERIFY command on the same vectors, every deployable model (88 configurations, three images) predicts the host class on 100 % of them, with outputs inside one quantisation step (int8) or 10^{-4} (fp32). The same models served by the team's bench firmware, built for the PC and driven over the serial protocol by the measurement harness, give the same answers (Fig. 1). The bundles for the real board compile with the pinned Arduino toolchain (`arduino:mbed_nano` 4.6.0, Chirale TFLM 2.0.0): 4 of 4 bundles link, at most 910 kB of the 960 kB application flash and 187 kB of static RAM, so the 80 bundled models are feasible.

These gates found three problems that would otherwise have reached the board unnoticed.

1) *Bands below float32 precision.* The first parity run failed on the two highest R2 bands (5.25–6 kHz), which lie in the stop-band of the 5 kHz low-pass. Their log-power is about ten decades below the signal bands and under the float32 FFT noise floor, so the C values are rounding noise. R2 now ends at 5.25 kHz (24 features; feature spec v1.1, contract rc3).
2) *Soft versus hard voting.* scikit-learn's forest averages leaf probabilities; the exported emlearn C code takes a majority vote. They disagree on 1.9 % of test windows on average (up to 32.7 % in one of 720 runs) and move

TABLE III
BEST CONFIGURATION PER FAMILY: MACRO-F1, MEAN ± STD OVER FOLDS AND SEEDS.

Family	P1 leaky	P2 recording	P3 bearing
DT	0.989±0.001	0.913±0.039	0.719±0.182
RF	0.997±0.001	0.962±0.054	0.742±0.256
MLP	1.000±0.000	0.949±0.074	0.653±0.137
CNN	1.000±0.000	0.955±0.061	0.594±0.176

macro-F1 by up to 0.19. All forest scores in this paper use the deployed rule.

3) *Optimised versus reference int8 kernels.* Desktop TFLite's default int8 kernels round differently from TFLM; int8 CNN outputs differed by up to 19 quantisation steps although TFLM with CMSIS-NN and with reference kernels agreed with each other exactly and with TFLite's reference kernels. On the full P2 fold-0 test set the two host paths agree on 92.9 % or more of windows (93 % for MLPs); macro-F1 changes by at most 0.035. The VERIFY reference now uses the reference kernels.

B. Leakage changes the answer (RQ3, accuracy)

Fig. 2 and Table III summarise 3,520 host runs. Under the leaky split every family looks near-perfect (best 1.000). Holding out a recording (P2) costs little for the best models but separates the families, and on unseen bearings (P3) the best model reaches only 0.742 macro-F1 (`rf-r4-t25d6-fp32`), on a 4-class task. Rankings move with the protocol: Kendall's τ between the per-model macro-F1 rankings is 0.42 (P1 vs P2), -0.20 (P1 vs P3) and 0.00 (P2 vs P3). Bearing-disjoint scores also vary strongly across folds and seeds (standard deviations up to 0.256 in Table III). In this sweep, the apparent P3 winner changed family when the MLPs went from one seed to five, so a single-seed P3 comparison is not reliable; every configuration here is reported over 5 seeds.

C. A physics-informed front end for unseen bearings (R4)

Without R4, no configuration exceeds 0.623 macro-F1 on unseen bearings (`rf-r2-t25d6-fp32`). The spectral representations describe the resonance band in absolute terms, and its level and shape change with fault size, the variable P3 holds out. R4 keeps only what envelope analysis looks at, the shape of the envelope spectrum around the fault impact rates. Table IV compares the best R4 configuration of each family with its best R1–R3 configuration. On unseen bearings R4 raises the best macro-F1 to 0.742 (`rf-r4-t25d6-fp32`); on the leaky and recording-wise splits it costs a few points, the same trade-off the protocol comparison shows: the features that fit the training recordings best are not the ones that transfer. The R4 front end needs 182.6 k estimated cycles on the virtual Cortex-M4 (R2: 124.0 k), and its C implementation matches the Python reference to 7.1×10^{-5}.

A recording fingerprint. The first R4 version kept all ten R1 statistics, and every R4 MLP scored about 0.339 macro-F1 on P2 fold 0 while the other folds stayed near 0.9. The cause is R1's first feature, the window mean: it is the accelerometer's DC offset, which differs from recording to recording, and one held-out recording lies 16.6 training standard deviations away from the rest. A tree only thresholds such a value; an MLP extrapolates on it. Without the mean, the same R4 MLPs average 0.899 on that fold. R4 drops the mean; R1–R3 keep it as frozen in the feature specification, so their MLP scores on P2 carry the same effect.

TABLE IV
BEST CONFIGURATION ON R1–R3 AGAINST BEST ON R4, PER FAMILY: MACRO-F1, MEAN ± STD OVER FOLDS AND SEEDS.

Family	Input	P1 leaky	P2 recording	P3 bearing
DT	R1–R3	0.989	0.902	0.584±0.216
	R4	0.951	0.913	0.719±0.182
RF	R1–R3	0.997	0.962	0.623±0.207
	R4	0.959	0.944	0.742±0.256
MLP	R1–R3	1.000	0.949	0.608±0.252
	R4	0.947	0.923	0.653±0.137

D. The front end dominates feature-based models (RQ2)

Fig. 3 separates the two phases on the virtual Cortex-M4. A decision tree or forest needs 277–1,589 estimated cycles per window, but its R1 front end needs 34.4 k cycles and its R2 front end (statistics plus a 1024-point FFT) 124.0 k: at least 95.7 % of the end-to-end work. R0 normalisation for CNNs costs 9.3 k cycles. Counting the front end therefore moves every feature-based model by two orders of magnitude on the energy axis, while CNNs barely move (Fig. 5). This is the effect that inference-only reporting hides.

E. Frontiers and family dominance (RQ1, RQ3)

Fig. 4 shows error against estimated end-to-end energy under P2 and P3. With three objectives (error, energy, peak RAM), the P2 frontier holds 7 models (3 with bootstrap membership probability ≥ 0.5) and the P3 frontier 6 (5). The bootstrap resamples test windows and draws each model's energy inside its cycle-model band (18 %–50 % wide), so membership reflects both accuracy noise and cost-model uncertainty. Ranked by hypervolume, the families order as RF $>$ DT $>$ MLP $>$ CNN under P1, RF $>$ DT $>$ MLP $>$ CNN under P2 and DT $>$ RF $>$ MLP $>$ CNN under P3 (Kendall τ = 1.00, 0.67 and 0.67 for P1–P2, P1–P3 and P2–P3).

F. Kernels, precision and memory

Table V lists the cost and accuracy of the best bearing-disjoint configuration of every family/input/precision group. On the same int8 CNNs, CMSIS-NN kernels cut estimated inference cycles by 5.9–31.4× (median 17.6×) relative to TFLM's reference kernels (Fig. 6); float CNNs, which CMSIS-NN does not accelerate, need 5.4–30.5× the cycles of their int8 versions. For the smallest MLPs, int8 brings no speed-up, because quantising the input and requantising each layer cost about as much as the few hundred MACs they replace. Peak RAM (arena, feature buffers and stack) ranges from 128 B to 102 kB, and the largest model needs 324 kB of flash.

tools/vmcu/run_vmcu.py — virtual Cortex-M4

```
$ python tools/vmcu/run_vmcu.py --images trees cmsis ref   QEMU mps2-an386 (Cortex-M4F) + vmcount plugin, arm-none-eabi-gcc 13.2
-Os, softfp ABI (as the Nano core)
```

Deployable models running as Cortex-M4F code (median of 5 stored windows)

model_id	runtime	flash B	arena B	stack B	front end cyc	inference cyc	latency ms @64MHz	VERIFY agree
dt-r4-d8-fp32	emlearn	710	0	384	182,606	277	2.86	100%
dt-r2-d4-fp32	emlearn	250	0	344	124,049	277	1.94	100%
dt-r1-d12-fp32	emlearn	1,942	0	88	34,442	283	0.54	100%
rf-r2-t25d10-fp32	emlearn	15,822	0	344	124,049	1,504	1.96	100%
rf-r1-t25d10-fp32	emlearn	25,106	0	88	34,442	1,565	0.56	100%
rf-r4-t25d10-fp32	emlearn	29,584	0	384	182,606	1,589	2.88	100%
mlp-r2-h64×64-int8	tflm-cmsis	12,568	2,132	860	124,049	52,927	2.77	100%
mlp-r4-h64×64-int8	tflm-cmsis	13,656	2,132	860	182,606	58,687	3.77	100%
mlp-r2-h64×64-fp32	tflm-cmsis	27,728	1,360	368	124,049	80,332	3.19	100%
mlp-r4-h64×64-fp32	tflm-cmsis	32,080	1,360	384	182,606	92,374	4.30	100%
mlp-r3-h128-int8	tflm-cmsis	39,888	2,212	860	145,936	205,774	5.50	100%
mlp-r3-h128-fp32	tflm-cmsis	138,440	2,208	368	145,936	385,893	8.31	100%
cnn-r0-w200d4-int8	tflm-cmsis	96,664	30,100	1,164	9,273	43,955,758	686.95	100%
cnn-r0-w200d4-int8qat	tflm-cmsis	96,920	30,004	1,164	9,273	43,955,792	686.95	100%
cnn-r0-w200d4-fp32	tflm-cmsis	331,768	100,096	448	9,273	1,339,635,018	20931.94	100%
cnn-r0-w200d4-int8ref	tflm-ref	96,664	29,220	1,448	9,273	1,380,326,159	21567.74	100%

```
(largest configuration of each family/input/precision group shown; all 88 models in results/vmcu/vmcu_metrics.csv; VERIFY = class
agreement with the host on the 100 test vectors)
VERIFY replay: 88/88 models agree with the host on ≥ 99 % of the vectors, outputs within one int8 step / 1e-4
M2 parity r1: max |C - Python| = 1.37e-05 on 100 vectors → PASS (gate < 1e-3)
M2 parity r2: max |C - Python| = 1.37e-05 on 100 vectors → PASS (gate < 1e-3)
M2 parity r3: max |C - Python| = 9.56e-05 on 100 vectors → PASS (gate < 1e-3)
M2 parity r4: max |C - Python| = 7.09e-05 on 100 vectors → PASS (gate < 1e-3)
```

(a) Virtual Cortex-M4: the deployable code of every model runs as Cortex-M4F machine code in QEMU (largest configuration of each group shown; all 68 in `results/vmcu/vmcu_metrics.csv`).

tools/demo_sim.py — simulated board

```
$ python tools/demo_sim.py   simulated Nano on /dev/pts/0  (bench_core.cpp built for PC)
> PING   → {'bench': 'sim-models', 'n_models': 20}
> LIST   → 20 models: dt-r1-d4-fp32, dt-r1-d6-fp32, ... cnn-r0-w025d2-int8qat
```

SEL k + VERIFY x 100 test vectors vs host outputs (CONTRACT.md 7.2)

k	model_id	runtime	arena B	class agree	max |Δout|	RUN 50 → cls
0	dt-r1-d4-fp32	emlearn	0	100%	0.0e+00	0
1	dt-r1-d6-fp32	emlearn	0	100%	0.0e+00	0
2	dt-r1-d8-fp32	emlearn	0	100%	0.0e+00	0
3	dt-r1-d12-fp32	emlearn	0	100%	0.0e+00	0
4	dt-r2-d4-fp32	emlearn	0	100%	0.0e+00	2
5	rf-r1-t5d6-fp32	emlearn	0	100%	0.0e+00	0
6	rf-r1-t5d10-fp32	emlearn	0	100%	0.0e+00	0
7	rf-r1-t10d6-fp32	emlearn	0	100%	0.0e+00	0
8	rf-r1-t10d10-fp32	emlearn	0	100%	0.0e+00	0
9	rf-r1-t25d6-fp32	emlearn	0	100%	0.0e+00	0
10	rf-r1-t25d10-fp32	emlearn	0	100%	0.0e+00	0
11	mlp-r3-h32-fp32	tflm-host	2224	100%	4.4e-06	0
12	mlp-r3-h32-int8	tflm-host	1776	100%	5.1e-07	0
13	mlp-r3-h64-fp32	tflm-host	2352	100%	2.4e-06	0
14	mlp-r3-h64-int8	tflm-host	2064	100%	5.1e-07	0
15	mlp-r3-h128-fp32	tflm-host	2608	100%	2.4e-06	0
16	mlp-r3-h128-int8	tflm-host	2640	100%	5.1e-07	0
17	cnn-r0-w025d2-fp32	tflm-host	14208	100%	1.7e-06	0
18	cnn-r0-w025d2-int8	tflm-host	5152	100%	7.4e-02	0
19	cnn-r0-w025d2-int8qat	tflm-host	5056	100%	1.0e-01	0

```
20/20 models agree with the host on ≥ 99 % of the 100 vectors  (4 s, 2000 VERIFY round trips over the serial
protocol)
```

(b) Simulated board: the team's bench firmware, built for the PC with real models and the C front end, driven over the serial protocol by `tools/dut.py` (SEL, VERIFY on the 100 test vectors, RUN).

Fig. 1. The two pre-hardware demonstrations. Both use the exact artefacts that will be flashed to the Nano 33 BLE.

G. Battery life (RQ4)

With chip-level sleep power (9.5 µW) and a CR2032 cell, the crossover rate r^*, below which sleep dominates and the model's energy stops mattering, ranges from 5×10^{-5} Hz (`cnn-r0-w200d4-fp32`) to 0.332 Hz (`dt-r4-d4-fp32`) for the most accurate P3 model of each family (Fig. 8). The most accurate CNN is a float model; its int8 version (`cnn-r0-w200d4-int8`) scores 0.592 against 0.594 for about 1/30 of the energy, a difference well inside the fold-to-fold spread, so an accuracy floor picks better than the single best score. At board-level sleep power (about 3 mW

TABLE V
PRE-HARDWARE COST ON THE VIRTUAL CORTEX-M4 (64 MHZ, ESTIMATED) AND HOST MACRO-F1 OF THE BEST P3 CONFIGURATION IN EACH FAMILY/INPUT/PRECISION GROUP.

Configuration	Flash (kB)	Peak RAM (kB)	Front end (kcyc)	Inference (kcyc)	Latency (ms)	Energy (μJ)	P1 F1	P2 F1	P3 F1
`dt-r1-d4-fp32`	0.3	0.1	34.4	0.3	0.54	5.4	0.640	0.631	0.547
`rf-r1-t5d6-fp32`	2.8	0.1	34.4	0.5	0.55	5.4	0.884	0.847	0.499
`dt-r2-d6-fp32`	0.4	9.5	124.0	0.3	1.94	19.4	0.957	0.804	0.584
`rf-r2-t25d6-fp32`	7.6	9.5	124.0	1.4	1.96	19.6	0.987	0.951	0.623
`mlp-r2-h64-int8`	6.1	11.4	124.0	26.4	2.35	23.5	1.000	0.905	0.607
`mlp-r2-h64-fp32`	10.4	10.5	124.0	32.2	2.44	24.4	1.000	0.905	0.608
`dt-r4-d4-fp32`	0.2	9.6	182.6	0.3	2.86	28.5	0.642	0.602	0.719
`rf-r4-t25d6-fp32`	9.0	9.6	182.6	1.6	2.88	28.7	0.936	0.915	0.742
`mlp-r4-h64-int8`	7.1	11.4	182.6	32.8	3.37	33.6	0.933	0.913	0.653
`mlp-r4-h64-fp32`	14.7	10.7	182.6	44.2	3.54	35.4	0.944	0.921	0.643
`mlp-r3-h64-int8`	20.6	12.5	145.9	114.0	4.06	40.6	1.000	0.935	0.434
`mlp-r3-h64-fp32`	68.4	12.3	145.9	196.5	5.35	53.4	1.000	0.933	0.435
`cnn-r0-w200d2-int8qat`	13.3	31.5	9.3	14443.0	225.82	2254.5	1.000	0.929	0.580
`cnn-r0-w200d4-int8`	94.4	34.5	9.3	43955.8	686.95	6858.5	1.000	0.921	0.592
`cnn-r0-w200d4-fp32`	324.0	102.2	9.3	1339635.0	20931.94	208984.5	1.000	0.949	0.594
`cnn-r0-w200d4-int8ref`	94.4	33.9	9.3	1380326.2	21567.74	215332.3	1.000	0.921	0.592

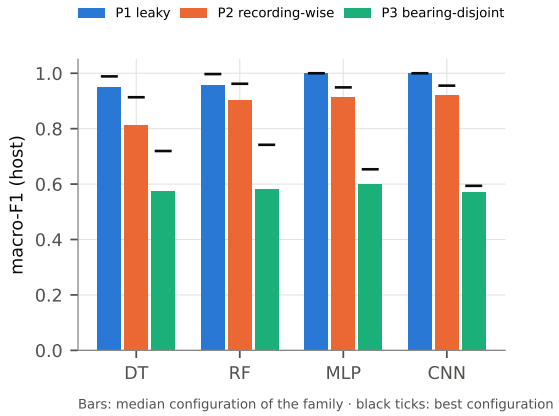


Fig. 2. Host macro-F1 per family under the three protocols (bars: median configuration; ticks: best configuration). Mean over all folds and 5 seeds.

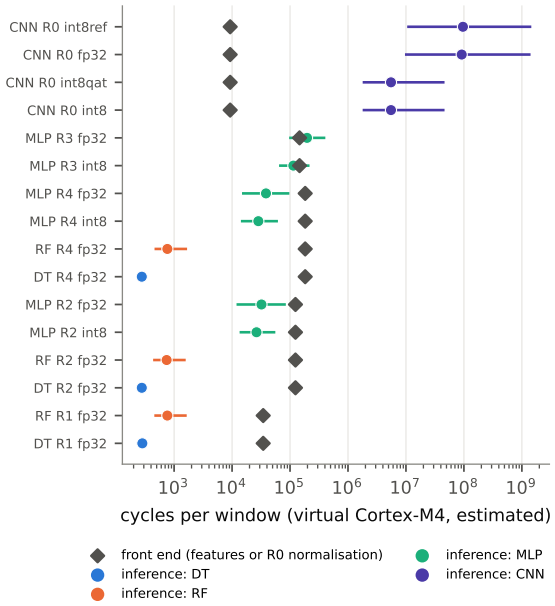


Fig. 3. Front-end versus inference cycles per window on the virtual Cortex-M4 (median over configurations; bars span the configurations of a group). Log scale.

for the Nano 33 BLE), sleep dominates at every practical rate. Measuring the board's real sleep floor is therefore part of the hardware campaign.

H. Robustness to sensor noise

Adding white noise to the test windows separates models that clean accuracy cannot. At 0 dB SNR under P2, the best configuration of each family keeps a macro-F1 of 0.283 (DT), 0.378 (RF), 0.359 (MLP) and 0.183 (CNN).

I. What the rig will resolve, and how long it will take

The virtual campaign (Section III-G, Fig. 7) gives a median session-to-session coefficient of variation of 0.46 % for the energy per inference with the 1 Ω shunt (1.3 % at the 95th percentile for the worst model), so the M3 gate (CV $<$ 3 %) is within reach, and two workloads whose power differs by 0.7 % can be told apart. The bias, at most 0.6 %, is mostly the INA219's 0.5 % gain error, which the known-load validation step calibrates. Precision is set by the stability of the board's baseline between the IDLE window and the window it is subtracted from, not by the INA219: changing the per-sample noise a hundredfold or the shunt from 0.1 to 1 Ω moves the error by tenths of a percent, whereas a baseline shift between 1 and 3 % takes the 95th-percentile CV past the gate. The contract's rule of an IDLE window immediately before every measured window is therefore the important one. The whole campaign for 88 models takes about 16 hours of board time, 10 of them in the LAT command (101 timed runs on 5 inputs), mostly for a few slow float and reference-kernel CNNs (`cnn-r0-w200d4-int8ref` alone needs 3.7 h). Capping LAT at about one minute per model (between 11 and 101

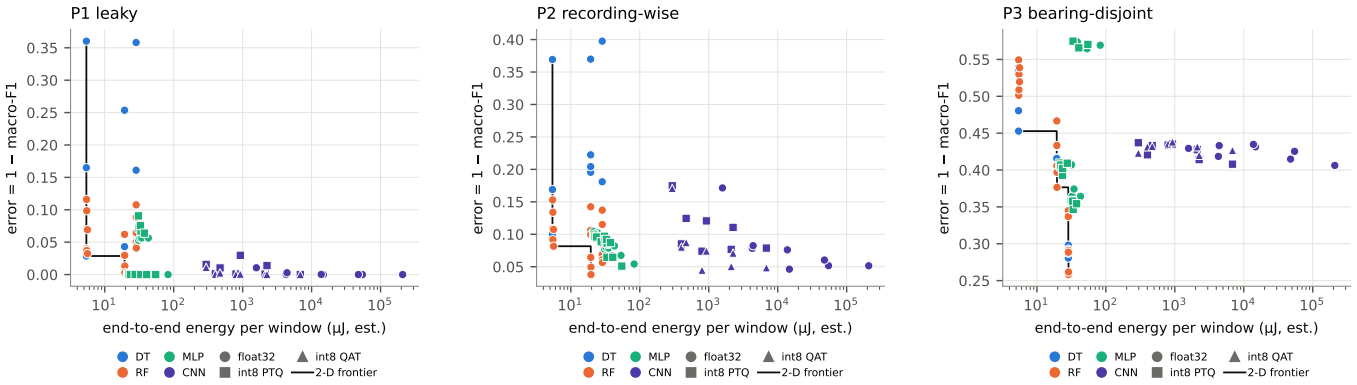


Fig. 4. Error versus estimated end-to-end energy per window for every deployable configuration under P1, P2 and P3. Colour: family; marker: precision; black staircase: two-dimensional frontier.

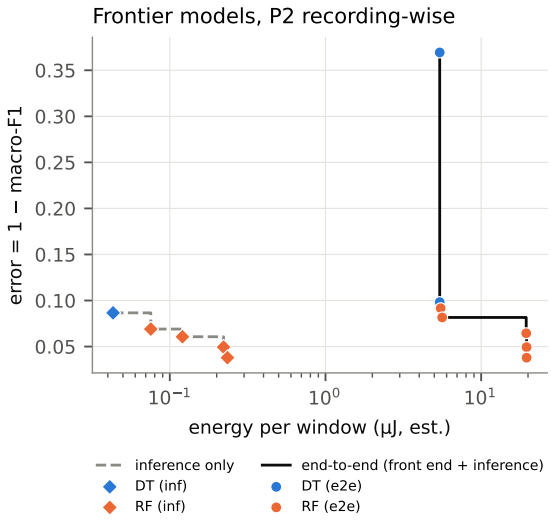


Fig. 5. Accuracy–energy frontier members under P2 with inference-only (diamonds, dashed) and end-to-end energy (circles, solid).

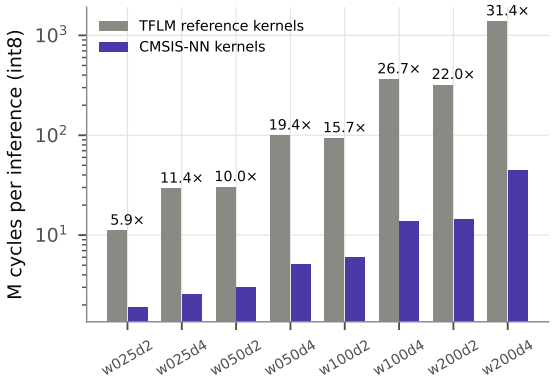


Fig. 6. Kernel ablation: estimated inference cycles of the int8 CNNs with TFLM reference kernels and with CMSIS-NN.

repetitions) brings the campaign to about 8 hours.

V. Toward a Selection Rubric

Fig. 9 gives the most accurate P3 model that fits each RAM and energy budget. It is the first version of the rubric the study will deliver. The procedure: fix RAM and flash budgets; derive an energy budget from the inference rate and the target battery life; if the rate is far below r^*, take the most accurate model that fits RAM; otherwise take the cheapest model above the accuracy floor. Always quote bearing-disjoint (P3) accuracy, never P1. The repository also publishes the rubric as an interactive page (https://agnibha11.github.io/TinyML-Pareto/selector/): given RAM and flash budgets, an inference rate, a battery and a target life, it returns the most accurate configuration that fits and the reason every other one does not.

As a worked example, take 64 kB of RAM, 256 kB of flash, a CR2032 cell and chip-level sleep. At one window per second, a two-year life leaves 29.0 µJ per window, and the rule picks `rf-r4-t25d6-fp32` (P3 macro-F1 0.742, 28.7 µJ). Asking for five years cuts the budget to 5.9 µJ, below the cost of the R4 front end alone, and the pick falls to `dt-r1-d4-fp32` (0.547). At one window per minute the budget is 1,741 µJ, the same forest fits with room to spare, and the cell lasts 7.7 years, close to the 8.1 years that sleep alone allows. The front end, not the classifier, is what the energy budget decides.

VI. Limitations and Next Steps

This version has no board measurements. Cycles come from an instruction-exact emulator with a static Cortex-M4 timing model. Energies are proportional to cycles, so they cannot show power differences between, for example, FPU- and integer-heavy code. The board's DWT counts (`LAT`) will calibrate the model, and `tools/vmcu/calibrate.py` is in place for that. The INA219 rig will then replace every energy number, including the board-level overheads (regulator, USB PHY) that the chip-level estimate omits. The virtual MCU uses current tflite-micro and GCC 13, while the board uses Chirale's TFLM build and GCC 7, a difference the calibration will quantify. CWRU has a single bearing per fault class and

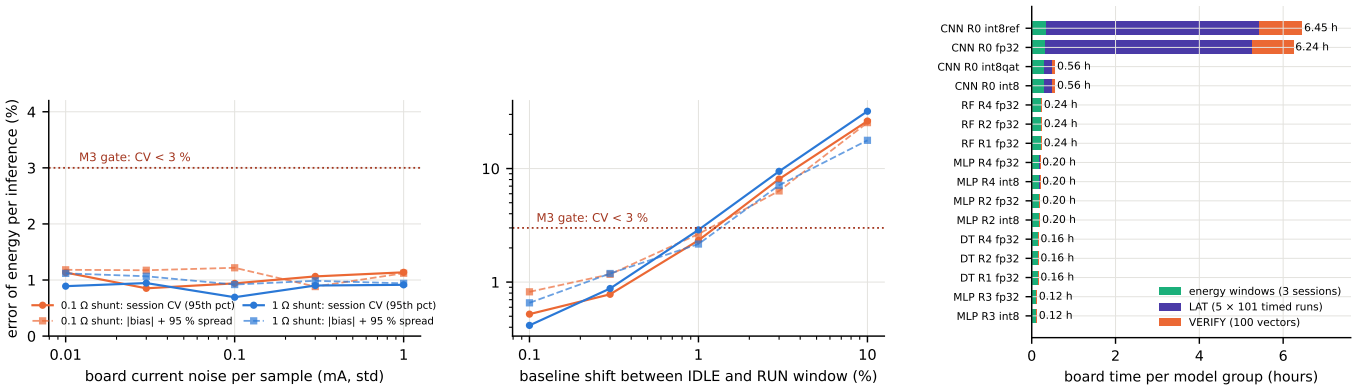


Fig. 7. Virtual measurement campaign. Left and centre: error of the energy per inference recovered from simulated INA219 windows (3 sessions, 30 replications) against board noise and against the baseline shift between the IDLE and RUN windows, for the 0.1 and 1 Ω shunts. Right: board time per model group.

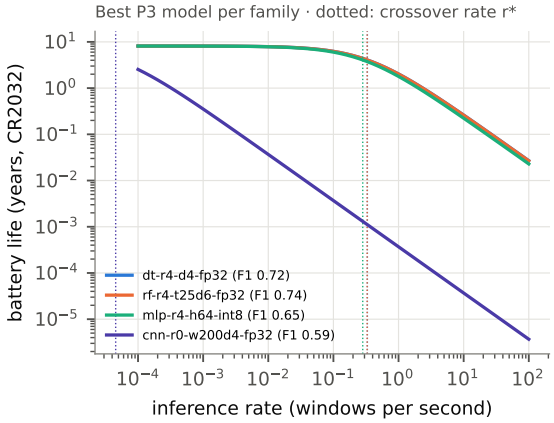


Fig. 8. Battery life versus inference rate for the most accurate P3 model of each family (estimated energy, chip-level sleep). Dotted: crossover rate r^*.

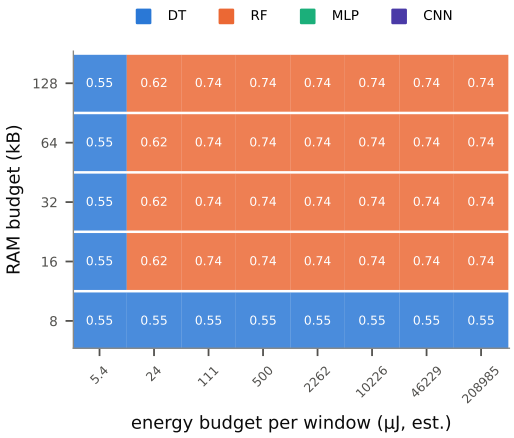


Fig. 9. Preliminary rubric: best P3 macro-F1 (cell value) and its family (colour) under RAM and estimated-energy budgets.

an artificially seeded fault, and in P3 the normal class can only be split by load. Paderborn is the planned generalisation check. R4's 2–5 kHz demodulation band was chosen for the CWRU test rig's resonances, and another machine may need a different band. The board noise, baseline drift and INA219 offsets in the virtual campaign are assumptions, not measurements of this board; the rig will measure them, and the campaign will be re-run with the measured values.

VII. Conclusion

Before any hardware arrived, the full pipeline was built and gated in emulation: data, leakage-checked splits, 88 deployable configurations, a virtual Cortex-M4 that runs the exact deployable code, and firmware bundles that link for the board. The preliminary answer to the project's question is already protocol-dependent. Leaky evaluation makes every family look interchangeable; bearing-disjoint evaluation does not. For feature-based models, the front end, not the classifier, sets the energy cost, and a physics-informed front end (R4) buys back much of the accuracy that leakage-free evaluation removes. The simulated campaign shows that the rig can resolve these effects if the board's baseline stays stable between windows. The hardware campaign now has to confirm them with measured energy.

References

[1] Z. Gao, Z. Jiang, Z. Dong, X. Fu, Y. Chen, and C. Zhang, "An edge-deployable TinyML approach enhanced by transfer learning for efficient bearing fault diagnosis," *Science China Technological Sciences*, vol. 68, p. 2220401, 2025.

[2] I. El Boughardini, H. Jebari, S. Rekiek, and K. Reklaoui, "Effective classification of bearing vibration signals using supervised machine learning for predictive maintenance: A lightweight 1D CNN for embedded deployment," *Journal Européen des Systèmes Automatisés*, vol. 59, no. 4, pp. 1141–1160, 2026.

[3] U. Muthumala, Y. Zhang, L. S. Martinez-Rau, and S. Bader, "Comparison of tiny machine learning techniques for embedded acoustic emission analysis," in *2024 IEEE 10th World Forum on Internet of Things (WF-IoT)*, 2024, pp. 444–449.

[4] J. Hendriks, P. Dumond, and D. A. Knox, "Towards better benchmarking using the CWRU bearing fault dataset," *Mechanical Systems and Signal Processing*, vol. 169, p. 108732, 2022.

[5] J. P. Vieira, V. A. Bauler, R. K. Rosa, and D. Silva, "Towards a more realistic evaluation of machine learning models for bearing fault diagnosis," *Mechanical Systems and Signal Processing*, vol. 258, p. 114640, 2026, arXiv:2509.22267.

[6] C. Banbury, V. J. Reddi, P. Torelli, N. Jeffries, C. Kiraly, J. Holleman, P. Montino, D. Kanter, P. Warden, D. Pau, U. Thakker, A. Torrini, J. Cordaro, G. Di Guglielmo, J. Duarte, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, "MLPerf tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021, arXiv:2106.07597.

[7] P. Jain, M. Kasper, G. Köber, O. Amft, A. Plinge, and D. Seuß, "Pareto optimal benchmarking of AI models on ARM Cortex processors for sustainable embedded systems," in *European Conference on EDGE AI Technologies and Applications (EEAI)*, 2025.

[8] P. Bartoli, C. Veronesi, A. Giudici, D. Siorpaes, D. Trojaniello, and F. Zappa, "Benchmarking energy and latency in TinyML: A novel method for resource-constrained AI," in *2025 International Joint Conference on Neural Networks (IJCNN)*, 2025, pp. 1–8.

[9] P. Bartoli, T. Bondini, C. Veronesi, A. Giudici, N. Antonello, and F. Zappa, "End-to-end efficiency in keyword spotting: A system-level approach for embedded microcontrollers," in *2025 IEEE SENSORS*, 2025, pp. 1–4.

[10] Y. Zhang, D. Wijerathne, Z. Li, and T. Mitra, "Power-performance characterization of TinyML systems," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*, 2022, pp. 644–651.

[11] S. Gupta, "Measured latency and energy of 144 INT8 bearing-fault networks on an ESP32-S3 with calibrated INA219 profiling," IEEE DataPort dataset, Oct. 2026.

[12] Case Western Reserve University Bearing Data Center, "Bearing data center," https://engineering.case.edu/bearingdatacenter, accessed Sep. 2026.

[13] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," *Mechanical Systems and Signal Processing*, vol. 64–65, pp. 100–131, 2015.

[14] R. B. Randall and J. Antoni, "Rolling element bearing diagnostics—A tutorial," *Mechanical Systems and Signal Processing*, vol. 25, no. 2, pp. 485–520, 2011.

[15] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 2005, pp. 41–46.

[16] J. Nordby, "emlearn: Machine learning inference engine for microcontrollers and embedded devices," 2019.

[17] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, arXiv:2010.08678.

[18] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," 2018, arXiv:1801.06601.

[19] *Cortex-M4 Technical Reference Manual, r0p1 (DDI 0439)*, Arm Ltd., 2010.

[20] *nRF52840 Product Specification v1.8*, Nordic Semiconductor, docs.nordicsemi.com, accessed Oct. 2026.

[21] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.